

BRAC^t's

Vishwakarma Institute of Technology, Pune

Practical Implementation Sheet

Department: CSE-AI Semester: 5 Academic Year: 2026-27

Division: E Batch: 2 Practical No: 3

Roll no:32 PRN No:12414914 Name of Student: Rashi Gundawar

Subject Name : Deep Learning

Problem Statement :

Implement forward propagation and backpropagation using TensorFlow/Keras. Analyze the effect of different learning rates and the number of epochs on model performance.

Algorithm :

1. Start.
2. Install and import the required libraries:
 - TensorFlow
 - Keras
 - Matplotlib
 - Pandas
3. Load the Fashion-MNIST dataset into training and testing sets.
4. Preprocess the data by:
 - Reshaping each 28×28 image into a 784-element vector.
 - Normalizing pixel values from 0–255 to 0–1.
5. Define a function to create a neural network model with:
 - Input layer of 784 neurons.

- First hidden layer of 128 neurons with ReLU activation.
 - Second hidden layer of 64 neurons with ReLU activation.
 - Output layer of 10 neurons with Softmax activation.
 - Adam optimizer and Sparse Categorical Crossentropy loss function.
6. Compare different learning rates (0.1, 0.01, 0.001):
- Create the model.
 - Train the model for 5 epochs.
 - Evaluate the model on the testing dataset.
 - Store the accuracy.
7. Display the learning rate and corresponding accuracy in a table.
8. Plot the graph of Learning Rate vs Accuracy.
9. Compare different numbers of epochs (5, 10, 20):
- Create the model with learning rate 0.001.
 - Train the model for the specified number of epochs.
 - Evaluate the model.
 - Store the accuracy.
10. Display the epoch-wise accuracy in a table.
11. Plot the graph of Epochs vs Accuracy.
12. Train the final model using:
- Learning rate = 0.001
 - Epochs = 20
13. Plot the Training Accuracy vs Validation Accuracy graph.
14. Plot the Training Loss vs Validation Loss graph.

15. Evaluate the final model on the testing dataset.

16. Display the Test Loss and Test Accuracy.

17. Stop.

Code :

```
# Install TensorFlow
!pip install -q tensorflow

import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.optimizers import Adam
import matplotlib.pyplot as plt
import pandas as pd

(x_train, y_train), (x_test, y_test) = tf.keras.datasets.fashion_mnist.load_data()
print("Training Shape:", x_train.shape)
print("Testing Shape :", x_test.shape)
x_train = x_train.reshape(-1, 784) / 255.0
x_test = x_test.reshape(-1, 784) / 255.0
print("After Normalization")
print("Training Data:", x_train.shape)
print("Testing Data :", x_test.shape)
```

```
# Model Creation
def create_model(lr):
    model = Sequential([
        Dense(128, activation='relu', input_shape=(784,)),
        Dense(64, activation='relu'),
        Dense(10, activation='softmax')
    ])

    model.compile(
        optimizer=Adam(learning_rate=lr),
        loss='sparse_categorical_crossentropy',
        metrics=['accuracy']
    )

    return model
```

```

# Learning Rate Comparison
learning_rates = [0.1, 0.01, 0.001]

results = []

for lr in learning_rates:
    model = create_model(lr)

    model.fit(
        x_train,
        y_train,
        epochs=5,
        batch_size=128,
        verbose=0
    )

    loss, acc = model.evaluate(
        x_test,
        y_test,
        verbose=0
    )

    results.append([lr, acc])

df = pd.DataFrame(results,
                   columns=["Learning Rate", "Accuracy"])

print(df)

plt.figure(figsize=(6,4))
plt.plot(df["Learning Rate"],
         df["Accuracy"],
         marker='o')

plt.xscale('log')
plt.title("Learning Rate vs Accuracy")
plt.xlabel("Learning Rate")
plt.ylabel("Accuracy")
plt.grid(True)
plt.show()

```

```

# Epoch Comparison
epochs_list = [5, 10, 20]

results_epoch = []

for ep in epochs_list:

    model = create_model(0.001)

    history = model.fit(
        x_train,
        y_train,
        validation_split=0.2,
        epochs=ep,
        batch_size=128,
        verbose=0
    )

    loss, acc = model.evaluate(
        x_test,
        y_test,
        verbose=0
    )

    results_epoch.append([ep, acc])

epoch_df = pd.DataFrame(results_epoch,
                        columns=["Epochs", "Accuracy"])

print(epoch_df)

plt.figure(figsize=(6,4))
plt.plot(epoch_df["Epochs"],
         epoch_df["Accuracy"],
         marker='o')

plt.title("Epochs vs Accuracy")
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.grid(True)
plt.show()

```

```
# Final Model Training
model = create_model(0.001)

history = model.fit(
    x_train,
    y_train,
    validation_split=0.2,
    epochs=20,
    batch_size=128
)

# Accuracy Graph
plt.figure(figsize=(6,4))
plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])
plt.title("Training vs Validation Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend(["Training", "Validation"])
plt.grid(True)
plt.show()

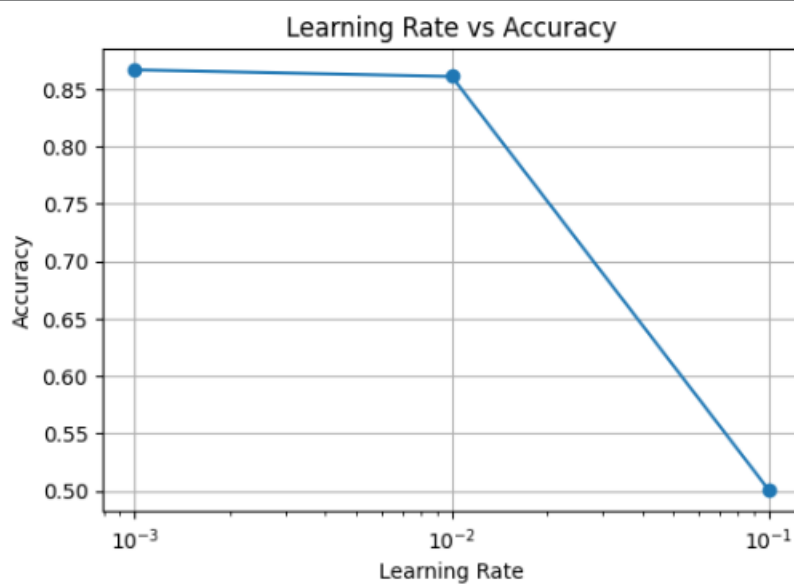
# Loss Graph
plt.figure(figsize=(6,4))
plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
plt.title("Training vs Validation Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend(["Training", "Validation"])
plt.grid(True)
plt.show()

# Final Evaluation
loss, accuracy = model.evaluate(x_test, y_test)

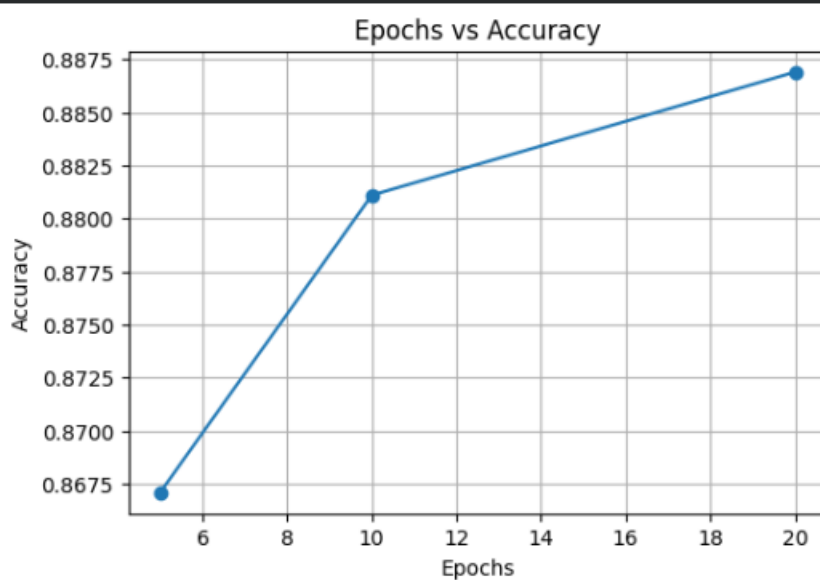
print("Test Loss :", loss)
print("Test Accuracy :", accuracy)
```

Output :

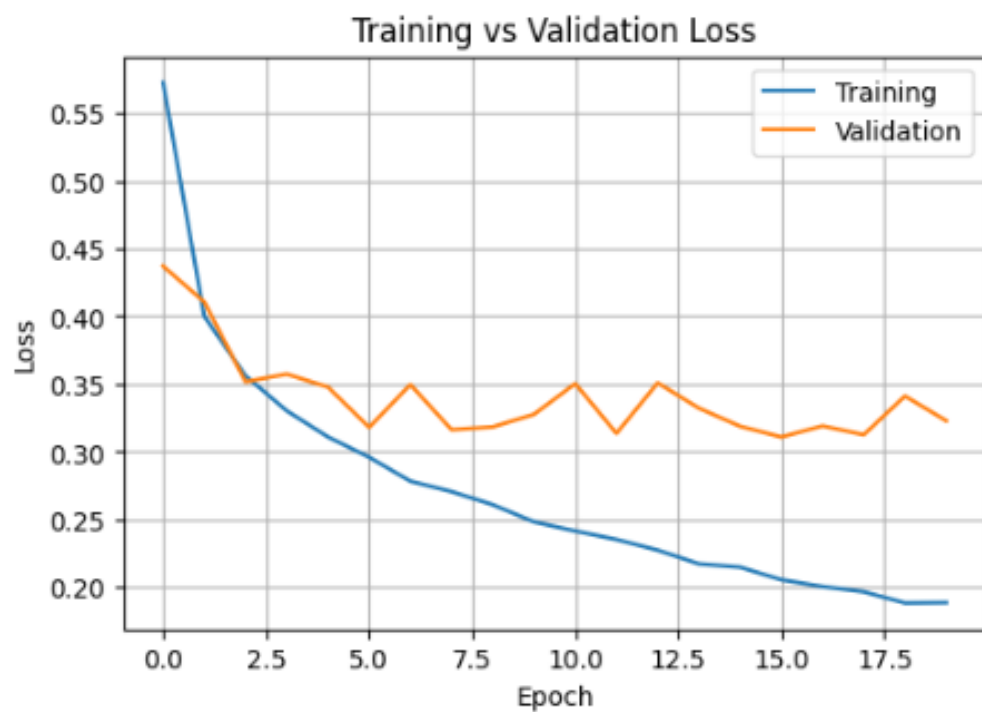
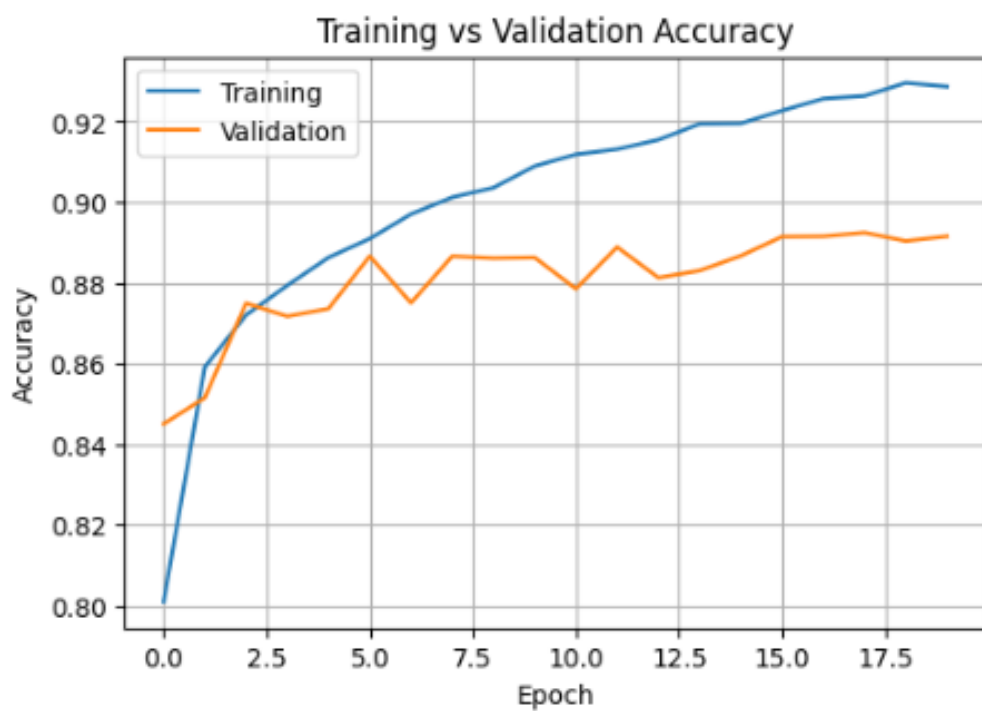
```
*** Training Shape: (60000, 28, 28)
Testing Shape : (10000, 28, 28)
After Normalization
Training Data: (60000, 784)
Testing Data : (10000, 784)
  Learning Rate  Accuracy
0      0.100    0.5007
1      0.010    0.8610
2      0.001    0.8667
```



```
Epochs  Accuracy
0        5    0.8671
1       10    0.8811
2       20    0.8869
```



```
*** Epoch 1/20
375/375 ————— 3s 6ms/step - accuracy: 0.8009 - loss: 0.5724 - val_accuracy: 0.8450 - val_loss: 0.4370
Epoch 2/20
375/375 ————— 2s 6ms/step - accuracy: 0.8591 - loss: 0.4000 - val_accuracy: 0.8515 - val_loss: 0.4106
Epoch 3/20
375/375 ————— 2s 5ms/step - accuracy: 0.8720 - loss: 0.3561 - val_accuracy: 0.8749 - val_loss: 0.3517
Epoch 4/20
375/375 ————— 2s 6ms/step - accuracy: 0.8793 - loss: 0.3303 - val_accuracy: 0.8717 - val_loss: 0.3574
Epoch 5/20
375/375 ————— 3s 8ms/step - accuracy: 0.8863 - loss: 0.3110 - val_accuracy: 0.8736 - val_loss: 0.3478
Epoch 6/20
375/375 ————— 4s 5ms/step - accuracy: 0.8909 - loss: 0.2961 - val_accuracy: 0.8866 - val_loss: 0.3181
Epoch 7/20
375/375 ————— 2s 5ms/step - accuracy: 0.8970 - loss: 0.2784 - val_accuracy: 0.8750 - val_loss: 0.3496
Epoch 8/20
375/375 ————— 2s 5ms/step - accuracy: 0.9012 - loss: 0.2707 - val_accuracy: 0.8866 - val_loss: 0.3163
Epoch 9/20
375/375 ————— 3s 8ms/step - accuracy: 0.9035 - loss: 0.2610 - val_accuracy: 0.8861 - val_loss: 0.3184
Epoch 10/20
375/375 ————— 4s 6ms/step - accuracy: 0.9089 - loss: 0.2484 - val_accuracy: 0.8863 - val_loss: 0.3277
Epoch 11/20
375/375 ————— 2s 5ms/step - accuracy: 0.9118 - loss: 0.2416 - val_accuracy: 0.8785 - val_loss: 0.3503
Epoch 12/20
375/375 ————— 2s 5ms/step - accuracy: 0.9131 - loss: 0.2354 - val_accuracy: 0.8888 - val_loss: 0.3136
Epoch 13/20
375/375 ————— 2s 5ms/step - accuracy: 0.9154 - loss: 0.2275 - val_accuracy: 0.8813 - val_loss: 0.3509
Epoch 14/20
375/375 ————— 4s 9ms/step - accuracy: 0.9194 - loss: 0.2174 - val_accuracy: 0.8830 - val_loss: 0.3322
Epoch 15/20
375/375 ————— 2s 6ms/step - accuracy: 0.9195 - loss: 0.2150 - val_accuracy: 0.8867 - val_loss: 0.3189
Epoch 16/20
375/375 ————— 2s 5ms/step - accuracy: 0.9226 - loss: 0.2057 - val_accuracy: 0.8914 - val_loss: 0.3110
Epoch 17/20
375/375 ————— 2s 5ms/step - accuracy: 0.9256 - loss: 0.2007 - val_accuracy: 0.8915 - val_loss: 0.3191
Epoch 18/20
375/375 ————— 2s 5ms/step - accuracy: 0.9263 - loss: 0.1968 - val_accuracy: 0.8924 - val_loss: 0.3126
Epoch 19/20
375/375 ————— 3s 9ms/step - accuracy: 0.9295 - loss: 0.1885 - val_accuracy: 0.8903 - val_loss: 0.3412
Epoch 20/20
375/375 ————— 4s 10ms/step - accuracy: 0.9286 - loss: 0.1888 - val accuracy: 0.8915 - val loss: 0.3230
```



313/313 ————— 1s 2ms/step - accuracy: 0.8805 - loss: 0.3604
Test Loss : 0.3603937029838562
Test Accuracy : 0.8805000185966492